If a sufficient amount of movement has been detected then we will launch our third function `getAnswer()` to generate a random message on the LCD. After that has completed, we pause for a half second to give the accelerometer a chance to calm down and then we make another reading and move the new values to the old reading just like we did in `setup()` to create a baseline for our sensor readings.

```
delay(125);
```

The last line in `loop()` is just a short eighth of a second delay to slow things down just enough. You probably have noticed that nowhere in this sketch do we know what the actual values of the sensor are. That's not really important only the amount of change, either smaller or larger, in the sensor value. So, let's look at our three new functions.

Functions

Of our three new functions, the first one that we've used is `getReading()`:

```
void getReading() {
  for (int i=0; i<3; i++) axis[i] = analogRead(axisPin[i]);
}
```

This function is a simple `for` loop that will read each analog pin as defined by the `axisPin[]` array and assign those readings to each of the 3 positions in the `axis[]` array. More on how that works briefly.

```
void oldReading() {
  for (int i=0; i<3; i++) oldAxis[i] = axis[i];
}
```

Like the last function, `oldReading()` uses a `for` loop to assign the current values in the `axis[]` array to the same index positions in the `oldAxis[]` array. This way we can keep track of the old readings so that we can detect when a sufficient amount of change has occurred.

```
void getAnswer() {
  int thisAnswer = random(40);
  while (thisAnswer % 2 != 0) thisAnswer = random(40);
```

Our final function has a lot going on beginning with choosing a random number in the range of 0 to 39. Even though we only have 20 answers, each answer is split into two lines so we want to start with an even number when displaying the first half of an answer. This means that we need to get 40 answers and then using a `while` loop, we can check to see if the random value is an odd or even number. By dividing the random number by 2, we know that we've got an even number when it is equal to 0. If it were an odd number we would start displaying our decision in the middle of our message so instead we have it try again at getting a random even number before moving on.

```
lcd.setCursor(0, 0);
lcd.print(allAnswers[thisAnswer]);
lcd.setCursor(0, 1);
lcd.print(allAnswers[thisAnswer+1]);
```

Once we have a number for our answer, these four lines will print the first line of the answer, move to the second line of the display, add one to our answer index, and then display the second line of our answer.